Listen

Week 09 Lab: System Tools, Threading & Unit Testing

What Are We Building?

Today you'll work with three skills that every Python developer needs: reading information about the machine your code runs on, running tasks in parallel with threads, and writing automated tests that prove your code works.

Open the starter files and look for the **`# TODO`** markers.

Q1: System Information Reporter

File: [`lab09_q1_starter.py`]

Ever wondered how tools like `neofetch` or "About This Mac" know what hardware and software you're running? Python's standard library gives you the same access. You'll build a mini dashboard that reports details about the OS, the Python interpreter, and a directory — all from three simple functions.

The starter already has the display helpers and error handling. **Your work is the three functions** that gather the data.

`get_system_info()` — use the `platform` module, return a dictionary:

```bash
"os" → platform.system()
"node" → platform.node()
"release" → platform.release()
"machine" → platform.machine()
```

`get_python_info()` — use the `sys` module, return a dictionary:

```bash
"version" → sys.version
"executable" → sys.executable
"platform" → sys.platform
```

`get_directory_info(path)` — use the `os` / `os.path` module, return a dictionary:

```bash
"path" → os.path.abspath(path)
"exists" → os.path.exists(path)
"file_count" → len(os.listdir(path)) if it exists, else 0
"is_directory" → os.path.isdir(path)
```

Reference

```python
# platform module — info about the operating system
platform.system()    # 'Darwin', 'Windows', or 'Linux'
platform.node()      # your computer's network name
platform.machine()   # 'arm64', 'x86_64', etc.

# sys module — info about the Python interpreter
sys.version          # '3.14.0 (main, ...)'
sys.executable       # '/usr/local/bin/python3'

# os module — info about directories
os.path.abspath(".") # full path to current directory
os.listdir(".")      # list of files in a directory
```

Test It

Run `python lab09_q1.py`. You should see something like:

```bash
============================================================
  SYSTEM INFORMATION REPORTER
============================================================

--- System Info ---
  os           : Darwin
  node         : Maziars-MacBook
```

```
  release      : 25.3.0
  machine      : arm64

--- Python Info ---
  version      : 3.14.0 (main, ...)
  executable   : /usr/local/bin/python3
  platform     : darwin

--- Directory Info for '.' ---
  path         : /Users/maziar/Downloads/maz_repo/testcomp/Week09
  exists       : True
  file_count   : 5
  is_directory : True

============================================================
```

Q2: Sequential vs Threaded Execution

File: [lab09_q2_starter.py]

Imagine making breakfast: you need to brew coffee (3 seconds), toast bread (2 seconds), and fry eggs (4 seconds). If you do them one at a time, that's 9 seconds of waiting. But in real life you'd start all three at once and be done in about 4 seconds — the time of the slowest task. That's exactly what threads do.

The starter already has the sequential runner and the timing/comparison code. **Your work is two functions** — the task simulator and the threaded runner.

`simulate_task(name, duration, lock)` — simulate one task:

```bash
Acquire the lock, print [START] {name}, release the lock
time.sleep(duration) — simulates real work
Acquire the lock, print [DONE]  {name} ({duration}s), release the lock
```

`run_threaded(tasks, lock)` — run all tasks at the same time:

```bash
Create an empty list called threads
```

```
For each (name, duration) in tasks, create a threading.Thread targeting
simulate_task with args=(name, duration, lock), and append it to threads
Start every thread (loop)
Join every thread (separate loop)
```

Reference

```python
import threading, time

# Creating and running a thread
t = threading.Thread(target=my_function, args=("hello", 5))
t.start()  # begins running in the background
t.join()   # wait for it to finish

# A lock prevents threads from printing over each other
lock = threading.Lock()
lock.acquire()
print("only one thread prints at a time")
lock.release()
```

Hints

- The lock makes sure two threads don't print on top of each other — without it, output gets jumbled.
- Use **two separate loops** — one to start, one to join. Starting and immediately joining in the same loop would make it sequential again.

Test It

Run `python lab09_q2.py`. You should see something like:

```bash
============================================================
  SEQUENTIAL vs THREADED EXECUTION
============================================================

--- Running SEQUENTIALLY ---
[START] Brew Coffee
[DONE]  Brew Coffee (3s)
[START] Toast Bread
[DONE]  Toast Bread (2s)
[START] Fry Eggs
[DONE]  Fry Eggs (4s)
Sequential time: 9.01 seconds
```

```
--- Running with THREADS ---
[START] Brew Coffee
[START] Toast Bread
[START] Fry Eggs
[DONE]  Toast Bread (2s)
[DONE]  Brew Coffee (3s)
[DONE]  Fry Eggs (4s)
Threaded time: 4.01 seconds

Speedup: 2.25x faster with threads!

============================================================
```

Q3: Unit Testing

File: [`lab09_q3_starter.py`]

So far in this course, you've been testing your code by running it and checking the output with your eyes. Unit testing automates that — you write small checks that Python runs for you, and it tells you exactly which ones pass or fail. In the real world, every serious project has hundreds of these tests.

Three small functions are already provided in the starter: `celsius_to_fahrenheit()`, `is_valid_ip()`, and `fizzbuzz()`. **Your work is writing the test classes** that verify each function behaves correctly.

`class TestCelsius(unittest.TestCase)` — write three test methods:

```javascript
test_freezing — celsius_to_fahrenheit(0) should equal 32.0
test_boiling — celsius_to_fahrenheit(100) should equal 212.0
test_negative — celsius_to_fahrenheit(-40) should equal -40.0 (the crossover point!)
```

`class TestValidIP(unittest.TestCase)` — write four test methods:

```javascript
test_valid — is_valid_ip("192.168.1.1") should be True
test_invalid_octet — is_valid_ip("256.1.1.1") should be False
test_too_few_parts — is_valid_ip("1.2.3") should be False
test_empty — is_valid_ip("") should be False
```

`class TestFizzBuzz(unittest.TestCase)` — write four test methods:

```javascript
test_fizz — fizzbuzz(3) should equal "Fizz"
test_buzz — fizzbuzz(5) should equal "Buzz"
test_fizzbuzz — fizzbuzz(15) should equal "FizzBuzz"
test_number — fizzbuzz(7) should equal "7"
```

Reference

```python
import unittest

class TestExample(unittest.TestCase):
    def test_addition(self):
        self.assertEqual(1 + 1, 2)

    def test_boolean(self):
        self.assertTrue("hello".startswith("h"))
        self.assertFalse("hello".startswith("x"))
```

Test It

Run `python lab09_q3.py`. You should see:

```bash
...........
----------------------------------------------------------------------
Ran 11 tests in 0.001s

OK
```

If a test fails, Python tells you exactly which one and why — that's the whole point.

Submission

- Only submit your GitHub (No Screenshot, no ZIP or Python file, just the Link) repository link, for example: **https://github.com/sojoudian/COMP2152_Labs**
- Create a branch named **`lab_week09`** , complete your lab work in that branch, and once finished, merge it into your **`master`** branch.